UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base – Dipartimento DIETI
Corso di Laurea Magistrale in Ingegneria Informatica

Adaptive Service Slicing & Multi-Stage DoS Mitigation in SDN

Architettura Cloud/Edge basata su OpenFlow 1.3, Hierarchical Token Bucket (HTB) e Macchina a Stati Adattiva

Docente: Prof. Giorgio Ventre
Insegnamento: Network and Cloud Infrastructures

Autori: Luigi Di Franco, Fabio Fusco
Anno Accademico: 2025/2026

Sommario

Il presente elaborato descrive la progettazione e la validazione sperimentale di un'architettura **Software-Defined Networking (SDN)** gerarchica di tipo Cloud/Edge, emulata in Mininet tramite switch **Open vSwitch (OVS)** nel datapath del kernel Linux e governata da un controller **Ryu** mediante il protocollo **OpenFlow 1.3**. L'infrastruttura risolve congiuntamente due problematiche critiche delle reti multi-tenant: (i) la garanzia deterministica della **Quality of Service (QoS)** tramite **Service Slicing** su link fisici congestionati mediante regolatori **Hierarchical Token Bucket (HTB)**, e (ii) la mitigazione autonoma di attacchi volumetrici e ad alto packet-rate di tipo **Denial of Service (DoS)**. Per minimizzare i falsi positivi verso i flussi legittimi ad alto throughput (*Elephant Flows*), il sistema adotta una telemetria asincrona disaccoppiata (*Zero-Backpressure*) e un automa a stati finiti (FSM) a tre stadi che combina policing tramite **OpenFlow Meter** (Stadio 1), isolamento mediante **Quarantine Hard Drop** sull'Edge switch (Stadio 2) e ripristino autonomo (**Auto-Healing**).

1 Introduzione e Obiettivi Architetturali

Nelle infrastrutture Cloud ed Edge Computing, la condivisione dei collegamenti di aggregazione tra flussi eterogenei espone la rete a due criticità fondamentali: la degradazione degli **SLA (*Service Level Agreement*)** dei servizi mission-critical durante le fasi di congestione causate da traffico *best-effort*, e la vulnerabilità ad attacchi volumetrici o di flooding (**DoS**) originati da nodi periferici compromessi.

L'architettura progettata persegue tre obiettivi principali:

1. **Disaccoppiamento rigoroso tra Control Plane e Data Plane:** l'inoltro, l'accodamento QoS, il policing e lo scarto dei pacchetti vengono eseguiti interamente nel datapath degli switch tramite primitive OpenFlow 1.3 e code kernel Linux, mantenendo il controller sollevato dal carico del piano dati dopo la classificazione del primo pacchetto.
2. **Service Slicing deterministico tramite HTB:** partizionamento di un link di uplink condiviso da $C = 25$ Mbps in due slice logiche (*Slice Gold* e *Slice Silver*) distinte a livello di trasporto (L4), garantendo che la banda minima contrattuale della Slice Gold (15 Mbps) sia sempre preservata anche in condizioni di sovraccarico (29 Mbps offerti).
3. **Mitigazione DoS progressiva Multi-Stage sull'Edge:** confinamento preventivo dei flussi anomali in uno stadio di *Policing* a 3.0 Mbps per discriminare i flussi legittimi pesanti (*Elephant Flows*) dagli attacchi persistenti, applicando il blocco hardware totale (*Hard Drop*) per 18 s solo in caso di saturazione continuativa per 6 s.

2 Topologia della Rete, Architettura SDN e Token Bucket

2.1 Topologia Gerarchica Edge/Core e Configurazione del Data Plane

L'infrastruttura emulata in Mininet utilizza switch **Open vSwitch (OVS)** operanti nel datapath nativo del kernel Linux e ricalca una topologia gerarchica Cloud/Edge a due livelli (Tabella **??**):

- **Livello Edge (Accesso Utenti):** composto dallo switch `s1`, che aggrega i client legittimi `h1` (*Slice Gold*, porta `s1-eth1`) e `h2` (*Slice Silver*, porta `s1-eth2`), e dallo switch `s2`, su cui è attestato il nodo malevolo `h3` (porta d'accesso `s2-eth1`).
- **Livello Core (Aggregazione e Cloud Server):** composto dallo switch `s3`, che raccoglie i flussi provenienti dagli uplink `s1-eth3` e `s2-eth2` e li inoltra verso i server Cloud `h4` (`10.0.0.100`, dedicato ai servizi Gold sulla porta L4 `5001`) e `h5` (`10.0.0.200`, dedicato ai servizi Silver sulle porte L4 `5002` e `5003`).

2.2 Architettura SDN e Pipeline Multi-Priorità OpenFlow 1.3

L'architettura separa nettamente il **Control Plane** (il controller centralizzato Ryu) dal **Data Plane** (gli switch OVS). All'avvio, gli switch contengono unicamente la regola *Table-Miss* (`priority=0`) che inoltra al controller il primo pacchetto sconosciuto di ogni flusso (`OFPPacketIn`). Il controller apprende dinamicamente la posizione degli host distinguendo le **porte di accesso** (su cui transita un solo indirizzo MAC sorgente) dalle **porte trunk** inter-switch (su cui transitano più MAC), localizzando così con precisione lo switch periferico di ciascun client.

Per classificare il traffico e applicare le difese direttamente nel kernel dello switch, Ryu installa regole `OFPFlowMod` strutturate su 6 livelli di priorità (Tabella 1).

Priorità	Funzione nella Pipeline	Campi di Match	Timeout	Azioni e Istruzioni OpenFlow 1.3
100	**Stadio 2: Quarantine Drop**	`IPv4`, `ipv4_src`, `ipv4_dst`	`hard = 18s`	`actions=[]` (Drop hardware immediato sull'Edge switch)
50	**Stadio 1: Meter Policing**	`IPv4`, `ipv4_src`, `ipv4_dst`	`idle = 30s`	`InstructionMeter(id)`, `SetQueue(3)`, `Output(port)`
20	**Service Slicing L4**	`IPv4`, `proto`, `src`, `dst`, `l4_dst`	`idle = 60s`	`SetQueue(1)` [Gold: `5001`] o `SetQueue(2)` [Silver: `5002/3`], `Output`
15	**Traffico ICMP**	`IPv4`, `ip_proto=1`, `src`, `dst`	`idle = 60s`	`Output(out_port)` (Passaggio diretto per diagnostica `ping`)
10	**Gestione ARP**	`eth_type=ARP`, `eth_dst`	`idle = 60s`	`Output(out_port)` (Prevenzione collisioni broadcast)
0	**Table-Miss**	*Wildcard* (`OFPMatch()`)	Permanente	`Output(OFPP_CONTROLLER)` (Inoltro a Ryu del primo pacchetto)

Tabella 1: Pipeline multi-priorità OpenFlow 1.3 progettata per lo Slicing e la mitigazione DoS.

2.3 I Token Bucket nel Progetto: Service Slicing (HTB) vs Policing (OpenFlow Meter)

Un regolatore **Token Bucket** è definito da due parametri: il tasso di rigenerazione dei token r [Mbps] e la capacità massima del secchio b [kbit] (*burst size*). Il volume massimo di dati $A(\Delta t)$ che può attraversare il regolatore in un intervallo di tempo Δt è limitato superiormente dalla relazione lineare:

$$A(\Delta t) \leq b + r \cdot \Delta t \tag{1}$$

All'interno del progetto, questo principio matematico è impiegato in due punti strategici con due modalità differenti:

1. **Token Bucket con Coda per il Service Slicing (Linux HTB su `s1-eth3`):** Il collegamento condiviso `s1-eth3` tra lo switch Edge `s1` e il Core `s3` è configurato tramite OVSDB come collo di bottiglia fisico con capacità totale $C_{\text{link}} = 25$ Mbps. Quando `h1` richiede $R_{\text{Gold}} = 15$ Mbps e `h2` richiede $R_{\text{Silver}} = 14$ Mbps, la domanda complessiva supera la capacità del cavo:

$$R_{\text{tot}} = R_{\text{Gold}} + R_{\text{Silver}} = 15 + 14 = 29 \text{ Mbps} > C_{\text{link}} \ (25 \text{ Mbps}) \tag{2}$$

   Il controller assegna i pacchetti di `h1` (porta `5001`) alla **Queue 1 (Slice Gold)** e quelli di `h2` (porta `5002`) alla **Queue 2 (Slice Silver)**. L'algoritmo HTB distribuisce i token dando priorità assoluta alla Queue 1 (Priorità 1, $r_{\text{min,Gold}} = 15$ Mbps, $r_{\text{max,Gold}} = 25$ Mbps) rispetto alla Queue 2 (Priorità 2, $r_{\text{min,Silver}} = 5$ Mbps, $r_{\text{max,Silver}} = 10$ Mbps). I throughput effettivamente trasmessi sul link risultano quindi:

$$B_{\text{Gold}} = R_{\text{Gold}} = 15 \text{ Mbps}, \qquad B_{\text{Silver}} = \min\left(R_{\text{Silver}},\ C_{\text{link}} - B_{\text{Gold}},\ r_{\text{max,Silver}}\right) = 10 \text{ Mbps} \tag{3}$$

   L'eccesso di 4 Mbps generato da `h2` viene messo in coda nel buffer della Queue 2 e scartato per *tail-drop*, preservando il 100% del traffico mission-critical di `h1`.
2. **Token Bucket senza Coda per la Mitigazione DoS (OpenFlow Meter su `s2`):** Quando un host viene posto nello Stadio 1 di mitigazione sull'Edge switch `s2`, il controller installa un **Meter OpenFlow 1.3** (`OFPMeterBandDrop`) con tasso $r_{\text{meter}} = 3000$ kbps (3.0 Mbps) e tolleranza $b_{\text{meter}} = 300$ kbit, associato alla `Queue 3` HTB ($r_{\text{max},3} = 3.0$ Mbps). Il Meter agisce come puro *Policer*: i pacchetti che trovano token nel secchio passano verso il Core, mentre i pacchetti che eccedono i 3.0 Mbps vengono scartati immediatamente nella pipeline dello switch.

3 Telemetria Asincrona e Macchina a Stati Anti-DoS

3.1 Telemetria Disaccoppiata su Due Punti Topologici

Il controller Ryu raccoglie le statistiche ogni $T_{\text{mon}} = 2.0$ s (con un anticipo di 0.3 s per compensare il tempo di risposta OpenFlow). Indicando con $\Delta B = B_{\text{now}} - B_{\text{prev}}$ e $\Delta P = P_{\text{now}} - P_{\text{prev}}$ i byte e i pacchetti transitati nell'intervallo Δt, il controller calcola per ogni flusso e porta il throughput R e il packet-rate Π:

$$R = \frac{\Delta B \times 8}{\Delta t \times 10^6} \ \text{[Mbps]}, \qquad \Pi = \frac{\Delta P}{\Delta t} \ \text{[pps]} \tag{4}$$

Per avere una visione non distorta dal taglio dei pacchetti, il monitoraggio avviene in due punti distinti:

- **Misura dell'Attacco Iniettato ($R_{\text{edge}}, \Pi_{\text{edge}}$ su `s2-eth1`):** ottenuta tramite `OFPPortStatsRequest` sui contatori hardware di ricezione (`rx_bytes`) della porta d'accesso dell'Edge switch `s2`. Misura quanto traffico l'host sta realmente immettendo sul cavo.
- **Misura del Traffico Consegnato (R_{core} su `s3`):** ottenuta tramite `OFPFlowStatsRequest` sul Core switch `s3` per le regole con priority ≥ 20. Misura il traffico effettivo che supera le difese Edge e raggiunge i Cloud Server.

3.2 Funzionamento della Macchina a Stati a 3 Stadi

Ad ogni indirizzo IP sorgente è associato uno stato $S \in \{0, 1, 2\}$ progettato per bloccare gli attacchi DoS evitando falsi positivi contro i flussi legittimi ad alto volume (*Elephant Flows*):

- **Stadio 0 – Normal:** il traffico scorre nelle normali code di Slicing (`priority=20`). Il controller valuta due condizioni di allarme sulla porta d'accesso Edge:

$$\begin{cases} \text{Se } R_{\text{edge}} > 21.0 \text{ Mbps} & \Longrightarrow \quad \text{Transizione allo } \textbf{Stadio 1 (Metering)} \\ \text{Se } \Pi_{\text{edge}} > 4500 \text{ pps} & \Longrightarrow \quad \text{Transizione diretta allo } \textbf{Stadio 2 (Hard Drop)} \end{cases} \tag{5}$$

- **Stadio 1 – Throttling & Metering (3.0 Mbps):** il controller installa su `s2` la regola a `priority=50` che vincola il flusso sospetto al Meter da 3.0 Mbps e inizializza il contatore di persistenza $c = 1$. Ad ogni ciclo successivo (2.0 s), il controller verifica se l'host continua a saturare la banda concessa:

$$R_{\text{edge}} \geq 2.2 \text{ Mbps} \quad \vee \quad R_{\text{core}} \geq 2.2 \text{ Mbps} \tag{6}$$

  Un flusso TCP legittimo (*Elephant Flow*), subendo lo scarto dei pacchetti oltre i 3 Mbps, riduce la propria finestra di congestione; se il rate scende sotto 1.0 Mbps, il controller rimuove il Meter e riporta l'host allo **Stadio 0**. Al contrario, se un attaccante UDP ignora il limite e satura il bucket per $N_{\text{max}} = 3$ cicli consecutivi:

$$T_{\text{persistenza}} = N_{\text{max}} \times T_{\text{mon}} = 3 \times 2.0 \text{ s} = 6.0 \text{ secondi} \tag{7}$$

  il controller decreta l'attacco DoS conclamato e innesca il passaggio allo **Stadio 2**.
- **Stadio 2 – Quarantine Drop e Auto-Healing:** viene installata sull'Edge switch `s2` una regola a massima priorità (`priority=100`) con lista di azioni vuota (`actions=[]`) e timer nativo `hard_timeout = 18s`. Tutti i pacchetti dell'attaccante vengono scartati in ingresso su `s2`, azzerando il carico verso il Core `s3`. Allo scadere dei $T_q = 18.0$ s:

$$t_{\text{fine}} = t_{\text{escalation}} + T_q = t_{\text{escalation}} + 18.0 \text{ s} \tag{8}$$

  lo switch rimuove autonomamente la regola di blocco e il controller ripristina l'host allo **Stadio 0 (Auto-Healing)**.

4 Analisi del Grafico Sperimentale e Giustificazione degli Andamenti

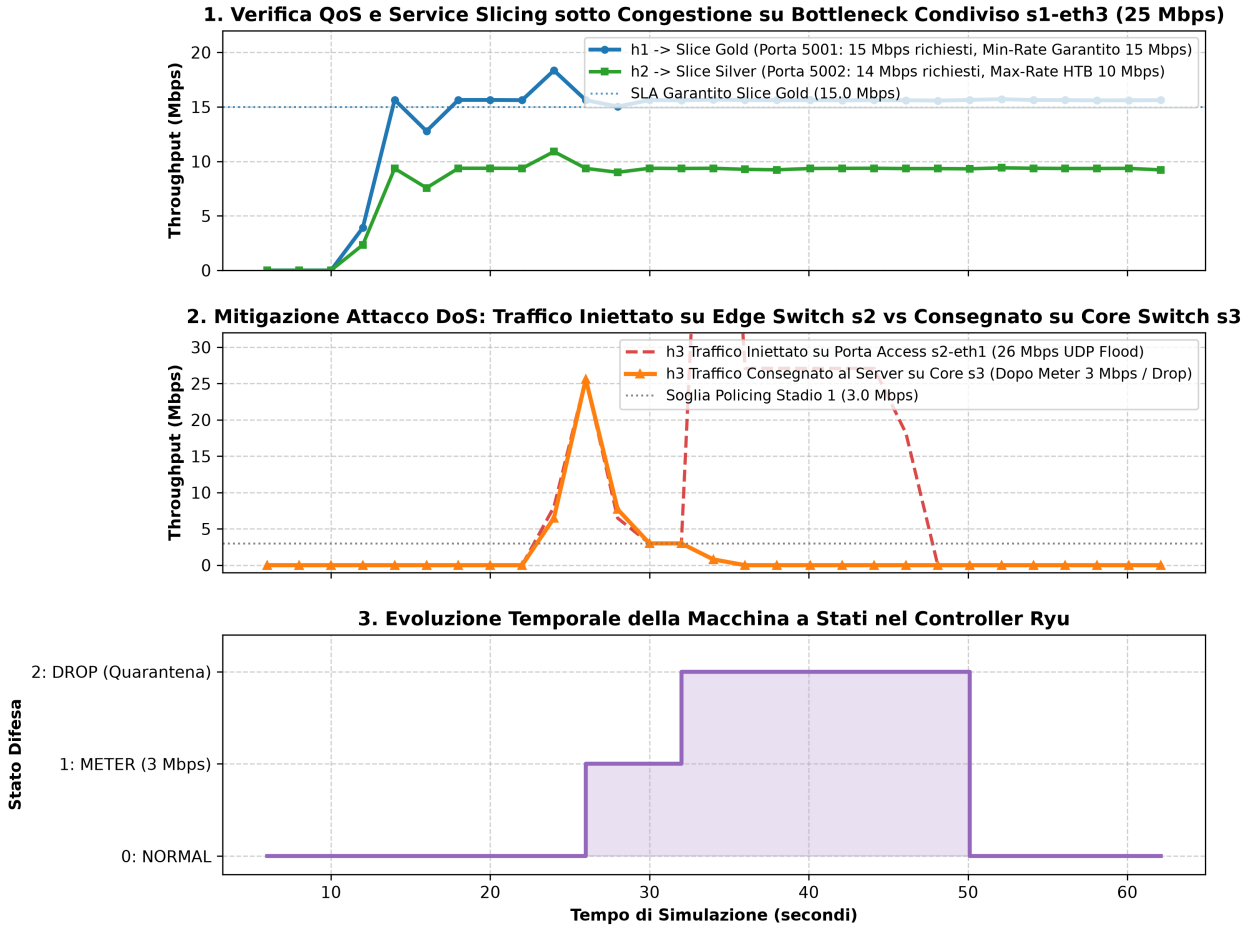


Figura 1: Analisi sperimentale: Service Slicing, mitigazione DoS e transizioni della Macchina a Stati.

I risultati della simulazione automatizzata (raccolti nel file `telemetry.csv` e rappresentati nei tre pannelli temporali di `risultati_dos_slicing.png` da $t = 6$ s a $t = 62$ s) confermano la piena efficacia dell'architettura progettata (Tabella 2).

4.1 Pannello 1: Verifica QoS e Service Slicing sotto Congestione (`s1-eth3`)

Nel primo grafico in alto sono tracciati i throughput consegnati al Core switch `s3` per l'utente Gold `h1` (curva blu con indicatori circolari) e per l'utente Silver `h2` (curva verde con indicatori quadrati):

- **Fase di setup ($t = 6 \dots 12$ s):** entrambe le curve sono a 0.0 Mbps mentre la rete completa la risoluzione ARP (`pingAll`) e l'Auto-Discovery delle porte Edge.
- **Attivazione della congestione e isolamento delle Slice ($t = 12 \dots 62$ s):** a $t = 12$ s partono contemporaneamente le trasmissioni UDP di `h1` (15 Mbps sulla porta `5001`) e `h2` (14 Mbps sulla porta `5002`), per un totale di 29 Mbps sul collo di bottiglia `s1-eth3` da 25 Mbps.
  - *Andamento di `h1` (Slice Gold):* dopo un breve assestamento iniziale dei contatori intorno a $t = 14$–24 s, la curva blu si posiziona stabilmente a $\sim$ **15.5 Mbps** (i 15.0 Mbps di payload UDP più l'overhead degli header Ethernet/IP/UDP), mantenendosi per tutta la prova esattamente sopra la linea tratteggiata dello **SLA Garantito (15.0 Mbps)**.
  - *Andamento di `h2` (Slice Silver):* la curva verde sale ma viene immediatamente bloccata dal Token Bucket della `Queue 2` HTB al suo valore di `max-rate`, stabilizzandosi in modo piatto a $\sim$ **9.5**–**10.0 Mbps**. I restanti 4 Mbps in eccesso vengono scartati da HTB senza causare alcuna perdita a `h1`.
  - *Immunità durante l'attacco DoS ($t = 24 \dots 50$ s):* osservando le curve blu e verde nella finestra centrale, non si registra alcuna flessione o oscillazione mentre `h3` attacca la rete, dimostrando l'isolamento completo tra i rami della topologia e le slice di servizio.

4.2 Pannelli 2 e 3: Mitigazione Attacco DoS ed Evoluzione della Macchina a Stati

Il secondo e il terzo pannello vanno letti congiuntamente: il Pannello 2 mette a confronto il **Traffico Iniettato da `h3` sulla porta d'accesso `s2-eth1`** (curva rossa tratteggiata) con il **Traffico Consegnato al Server sul Core `s3`** (curva arancione continua con triangoli), mentre il Pannello 3 mostra l'evoluzione a gradini dello **Stato Difesa** nel controller Ryu (curva viola):

1. **Innesco dell'attacco e passaggio allo Stato 1 ($t = 22 \dots 26$ s):** a $t = 22$ s l'host `h3` avvia l'UDP Flood a 26 Mbps verso `h5:5003`. Poiché `h3` parte dallo **Stato 0: NORMAL**, nei primi istanti il traffico attraversa `s2` e raggiunge `s3`: sia la curva rossa che quella arancione salgono fino al picco di **25.8 Mbps** registrato a **t = 26 s**. In questo esatto istante il controller rileva il superamento della soglia ($25.8 > 21.0$ Mbps) e nel Pannello 3 la curva viola commuta verticalmente dallo **Stato 0** allo **Stato 1: METER (3 Mbps)**.
2. **Finestra di Policing a 3.0 Mbps nello Stato 1 ($t = 26 \dots 32$ s):** non appena il controller installa la regola a `priority=50` con il Meter OpenFlow e la `Queue 3`, la curva arancione del traffico consegnato al Core `s3` crolla drasticamente da 25.8 Mbps, passando per il valore di transizione a $t = 28$ s (~ 7.8 Mbps) e appiattendosi nei campioni **t = 30 s** e **t = 32 s** esattamente a **3.00 Mbps**, sovrapponendosi alla linea grigia punteggiata della soglia di Policing.
3. **Escalation allo Stato 2 (Quarantine Drop) e azzeramento sul Core ($t = 32 \dots 50$ s):** durante i 3 campioni consecutivi nello Stato 1 ($t = 28, 30, 32$ s, pari a 6 secondi), il controller verifica che `h3` non ha ridotto la trasmissione ma continua a saturare i 3.0 Mbps concessi (≥ 2.2 Mbps). Pertanto, a **t = 32 s** la macchina a stati nel Pannello 3 sale al gradino **2: DROP (Quarantena)**:
   - *Comportamento della Curva Arancione (Traffico Consegnato su `s3`):* grazie alla regola a `priority=100` (`actions=[]`) installata su `s2`, la curva arancione scende a **0.00 Mbps** già a $t = 34$ s e rimane inchiodata a zero per tutto il resto della simulazione. Il Core switch `s3` e il server `h5` sono totalmente protetti.

- *Giustificazione fisica dell'andamento della Curva Rossa Tratteggiata ($t = 28 \ldots 48$ s):* osservando la curva rossa tratteggiata nel Pannello 2, si nota che tra $t = 28$ s e $t = 32$ s anch'essa scende a 3.0 Mbps, per poi risalire bruscamente sopra i 30 Mbps appena scatta lo Stato 2 ($t = 34$–42 s) prima di spegnersi a $t = 48$ s. Questo fenomeno è dovuto all'architettura del kernel Linux in Mininet: durante lo **Stadio 1**, i pacchetti di `h3` entrano in `s2` e vengono accodati nella `Queue 3` HTB (3 Mbps) della porta d'uscita `s2-eth2`; quando tale coda si riempie, il kernel Linux applica una contropressione locale (*backpressure* lungo il collegamento virtuale `veth`) che frena l'uscita dei datagrammi dall'interfaccia di `h3` a 3 Mbps. Nell'istante $t = 32$ s in cui subentra lo **Stadio 2 (Hard Drop)**, lo switch `s2` inizia a distruggere i pacchetti immediatamente in ingresso senza più passarli alla coda lenta `s2-eth2`: venendo a mancare il "tappo" di accodamento, il socket di `h3` scarica istantaneamente i pacchetti accumulati nel buffer generando il picco visibile sulla porta d'accesso `s2-eth1`, mentre oltre lo switch `s2` (curva arancione) non passa assolutamente nulla (0.0 Mbps).

4. **Scadenza dell'`hard_timeout` e Auto-Healing ($t = 50$ s):** esattamente 18 secondi dopo l'entrata in quarantena (32 s $+ 18$ s $=$ **50 s**), il timer hardware `hard_timeout=18` rimuove la regola a priorità 100 dallo switch `s2`. Nel Pannello 3 la curva viola scende verticalmente da **2: DROP** a **0: NORMAL** a $t = 50$ s, completando il ciclo di **Auto-Healing** e riportando l'infrastruttura nello stato nominale.

Intervallo	Stato FSM	`h1` Gold	`h2` Silver	`h3` Consegnato	Evento Chiave Osservato nei 3 Pannelli del Grafico
6–12 s	`0: NORMAL`	0.0 Mbps	0.0 Mbps	0.0 Mbps	Setup iniziale della rete, ARP learning e scoperta porte Edge
12–24 s	`0: NORMAL`	~ 15.5 Mbps	~ 9.5 Mbps	0.0 Mbps	Congestione su `s1-eth3` ($29 > 25$ Mbps): Gold garantito, Silver tagliato a 10 Mbps
24–26 s	`0` $\rightarrow$ `1`	~ 15.5 Mbps	~ 9.5 Mbps	25.8 Mbps	Picco UDP di `h3` a 25.8 Mbps (> 21 Mbps): attivazione immediata Stadio 1
26–32 s	`1: METER`	~ 15.5 Mbps	~ 9.5 Mbps	**3.0 Mbps**	Curva arancione limitata dal Meter a 3.0 Mbps per 3 cicli consecutivi (6 s)
32–50 s	`2: DROP`	~ 15.5 Mbps	~ 9.5 Mbps	**0.0 Mbps**	Escalation a Quarantine Drop (`prio=100`): curva arancione azzerata sul Core `s3`
50–62 s	`0: NORMAL`	~ 15.5 Mbps	~ 9.5 Mbps	0.0 Mbps	**Auto-Healing** a $t = 50$ s (dopo 18 s di drop): ritorno autonomo allo Stato 0

Tabella 2: Riepilogo temporale delle misure e delle transizioni di stato osservate nel grafico sperimentale.

5 Conclusioni

Il progetto realizzato dimostra come l'integrazione tra il paradigma **SDN (OpenFlow 1.3)** e il meccanismo **Hierarchical Token Bucket (HTB)** consenta di gestire efficacemente sia la Quality of Service sia la sicurezza in una rete Cloud/Edge. I risultati sperimentali confermano che il Service Slicing preserva costantemente i 15 Mbps contrattuali della Slice Gold anche sotto congestione, mentre la macchina a stati adattiva intercetta gli attacchi DoS in 2 secondi, discrimina i flussi tramite il policing a 3.0 Mbps per 6 secondi e isola completamente il Core della rete durante i 18 secondi di quarantena prima del ripristino automatico (*Auto-Healing*).